קטע קוד A

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.   static Groups groups;
 6.   Groups::iterator itr;
 7.   static Groups::iterator createGroup(const std::string& s) {
 8.     // see explanation regarding map::insert at the end
 9.     return groups.insert({s, 1}).first;
10.   }
11.   static Groups::iterator joinGroup(Groups::iterator group) {
12.     ++(group->second);
13.     return group;
14.   }
15.   static Groups::iterator joinGroup(const std::string& s) {
16.     auto itr = groups.find(s);
17.     if(itr == groups.end()) {
18.       itr = createGroup(s);
19.     }
20.     else {
21.       ++(itr->second);
22.     }
23.     return itr;
24.   }
25.   static void leaveGroup(Groups::iterator itr) {
26.     if(--(itr->second) == 0) {
27.       groups.erase(itr);
28.     }
29.   }
30.   class CharProxy {
31.     friend class MyString;
32.     Groups::iterator& itr;
33.     size_t index;
34.     CharProxy(Groups::iterator& itr, size_t i)
35.       : itr(itr), index(i) {}
36.   public:
37.     operator char() const { return itr->first[index]; }
38.     char operator=(const CharProxy& c) {
39.         return *this = static_cast<char>(c);
40.     }
41.     char operator=(char c) {
42.       std::string s = itr->first;
43.       leaveGroup(itr);
44.       s[index] = c;
45.       itr = joinGroup(s);
46.       return c;
47.     }
48.   };
```

```
public:
  MyString(const char* s = "") {
    itr = joinGroup(s);
  }
  ~MyString() { leaveGroup(itr); }
  MyString(const MyString& s) { itr = joinGroup(s.itr); }
  MyString& operator=(const MyString& s) {
    leaveGroup(itr); itr = joinGroup(s.itr); return *this;
  }
  char operator[](size_t index) const {
    return itr->first[index];
  }
  CharProxy operator[](size_t index) {
    return {itr, index};
  }
  friend ostream& operator<<(ostream& out, const MyString& a) {
    return out << a.itr->first << " (" << a.itr->second << ")";
  }
};

// in MyString.cpp
Groups MyString::groups;

// explanation regarding map::insert:
// map.insert returns std::pair<iterator, bool>
// 'first' is an iterator pointing to the new item or to
// an item with the same key that already existed, which
// prevented the insertion
// 'second' is a bool indicating whether insertion was performed

int main() {
    const MyString s1 = "Hello";
    MyString s2 = "World";
    MyString s3 = "hello";
    s2 = "Hello";
    s3[0] = 'H';
    cout << s1 << endl;
    s2 = "bye";
    // s3[1] = s2[1];
    cout << s1 << endl;
    cout << s1[1] << s2[1] << s3[1] << endl;
}
```

קטע קוד B

```
std::pair<bool, std::map<size_t, size_t>>
canFill(size_t big_bucket,
        const std::vector<size_t>& small_buckets,
        size_t index) {
  if(big_bucket == 0) return {true, {}};
  if(big_bucket < small_buckets.back()) return {false, {}};
  auto curr = small_buckets[index];
  if(big_bucket % curr == 0)
    return {true, { {curr, big_bucket / curr} }};
  if(index < small_buckets.size() - 1) {
     auto times = big_bucket / curr + 1;
     do {
       --times;
       auto rest = big_bucket - times * curr;
       auto result = canFill(rest, small_buckets, index + 1);
       if(result.first) {
         result.second[curr] = times;
         return {true, result.second};
       }
     } while(times > 0);
  }
  return {false, {}};
}

void printCanFill(size_t big_bucket,
                  const std::vector<size_t>& small_buckets) {
  cout << "Trying to fill " << big_bucket << " with: ";
  for(auto small: small_buckets) {
    cout << small << ' ';
  }
  auto result = canFill(big_bucket, small_buckets, 0);
  if(result.first) {
    cout << "- can fill!!" << std::endl;
    for(const auto& fill_bucket : result.second) {
      cout << "bucket: " << fill_bucket.first
           << ", times: " << fill_bucket.second << endl;
    }
  }
  else cout << "- cannot fill :-(" << endl;
}

int main() {
  // try to fill big bucket of size 10 with: 4
  printCanFill(10, {4});
  // try to fill big bucket of size 11 with: 7, 5, 4
  printCanFill(11, {7, 5, 4});
}
```

קטע קוד C – שאלות בונוס

```
template<class T, size_t SIZE>
class Stack {
  T arr[SIZE] = {};
  int pos = 0;
public:
  Stack& push(const T& t) {
    arr[pos++] = t;
    return *this;
  }
  Stack& push(T&& t) {
    arr[pos++] = std::move(t);
    return *this;
  }
  T pop() {
    return std::move(arr[--pos]);
  }
};

class Foo {
  std::string s;
public:
  Foo(const char* s = "@"): s(s) {}
  Foo(Foo&& foo): s(std::move(foo.s)) {}
  Foo& operator=(Foo&& foo) {
    s = std::move(foo.s);
    return *this;
  }
  void print() const { std::cout << s << std::endl; }
};

int main() {
  Stack<std::string, 5> s1;
  s1.push("hi").push(std::string{"bye"});
  std::cout << s1.pop() << std::endl;

  // Stack<Foo, 5> s2;
  // s2.push({});
  // Foo f = s2.pop();
  // f.print();

  // Stack<Foo, 5> s3;
  // Foo f2;
  // s3.push(f2);
}
```

בהצלחה !